ELDERCARE CONNECT

SMART ELDERLY ASSISTANCE PLATFORM

A Full Stack Web Application using the MERN Stack

(MongoDB, Express.js, React, Node.js)

Project Report submitted in partial fulfilment of the requirements for the course

Full Stack Web Development [FSD / SEM 5]

B. Tech Artificial Intelligence & Data Science — [Semester V / 2026]

Submitted by

Team Member Name	Roll Number	Role in Project
Santhosh S	727722EUAI052	Full Stack Architect, Backend & Deployment
[Team Member 2]	[Roll Number 2]	Frontend React & UI/UX Design
[Team Member 3]	[Roll Number 3]	Database Design, Mongoose & QA Automation

Under the guidance of

[Faculty Guide Name, Designation]

Department of Artificial Intelligence and Data Science

Kumaraguru College of Technology, Coimbatore — 641049

November, 2026

Abstract

ElderCare Connect is a full-stack smart assistance platform engineered to address the critical social and logistical vulnerabilities experienced by senior citizens living independently. As traditional multi-generational family structures evolve, elders frequently face acute barriers in obtaining dependable local assistance for essential daily chores, tracking multifaceted chronic medication regimens, monitoring vital biometric trends, and dispatching distress notifications during health emergencies. ElderCare Connect overcomes these challenges by establishing an integrated, role-based ecosystem linking four distinct user groups: Elderly Users, Family Members, Certified Volunteers, and System Administrators. Developed using the complete MERN technology stack—incorporating a responsive React 18 single-page frontend styled with Tailwind CSS, a robust Node.js and Express.js REST API server, and a high-availability MongoDB document datastore—the platform delivers a seamless, accessible user experience. The system features a location-based volunteer matching algorithm driven by the spherical Haversine formula, which automatically assigns verified volunteers within a 100-meter radius while routing out-of-range requests to an Administrator Manual Assignment Queue for human review. To guarantee inclusivity across diverse linguistic backgrounds, ElderCare Connect incorporates complete, real-time bilingual localization in English and Tamil (தமிழ்), alongside Web Speech API voice synthesis, automated node-cron medicine dosage alerts, Twilio SMS notifications, and an instantaneous one-touch emergency SOS dispatch pipeline. The application has been fully deployed on Render with cloud database persistence, establishing an enterprise-grade, compassionate community infrastructure.

Keywords: MERN Stack, Elderly Care, Location-Based Volunteer Assignment, Haversine Formula, Emergency SOS, Bilingual Localization, Node-cron Automation, Twilio SMS.

Chapter 1: Introduction

1.1 Background

The global demographic landscape is experiencing a profound transition characterized by unprecedented growth in the elderly population. According to public health data, individuals aged 60 and older represent the fastest-growing demographic cohort globally. Concurrently, rapid urbanization, domestic and international professional migration, and changing social dynamics have significantly disrupted conventional family support systems. Millions of senior citizens now live independently, separated by geographical distances from their adult children and immediate relatives.

While independent living fosters personal dignity, it exposes seniors to significant physical, cognitive, and logistical challenges. Routine activities such as procuring prescription medications, scheduling clinical consultations, purchasing groceries, and managing chronic health vitals become intimidating hurdles. Most critically, acute medical emergencies such as falls, cardiac distress, or sudden loss of mobility demand immediate coordinated intervention from trusted family members, emergency medical services, and nearby community volunteers. Modern digital health platforms often fail elderly demographics due to convoluted user interfaces, complex authentication barriers, lack of localized language support, and absence of physical community integration.

1.2 Problem Statement

Existing commercial elderly care services and mobile applications are fragmented, prohibitively expensive, and fundamentally misaligned with the usability requirements of senior citizens. Senior citizens who live independently face severe logistical vulnerabilities when seeking timely, trusted assistance for daily errands and emergency scenarios. Conventional platforms lack intelligent, distance-aware volunteer dispatch mechanisms, often leading to unfulfilled requests or dangerous delays during crises. Furthermore, digital tools predominantly cater to English-speaking, tech-savvy users, alienating millions of vernacular-speaking seniors who lack intuitive voice or localized interfaces. There is an urgent societal need for an integrated, accessible, and automated platform that unifies seniors, family members, local volunteers, and administrators within a cohesive, bilingual support network.

1.3 Objectives

The primary objective of ElderCare Connect is to architect, develop, test, and deploy a production-ready, accessible web platform for smart elderly assistance. Specifically, the measurable objectives are:

- **Objective 1 (Secure Role-Based Access Control):** Implement JSON Web Token (JWT) authentication and bcrypt password hashing across four explicit user roles: Elderly User, Family Member, Community Volunteer, and System Administrator.
- **Objective 2 (Smart Location-Based Volunteer Assignment):** Develop an automated dispatch engine utilizing the Haversine spherical distance metric that automatically assigns available volunteers within a 100-meter radius, while enforcing an Administrator Manual Assignment Queue for requests exceeding 100 meters.
- **Objective 3 (Rapid Emergency SOS Dispatch):** Engineer a one-touch emergency SOS triggering pipeline integrated with live Socket.IO broadcasting, Twilio SMS automated alerts, and family contact notifications.
- **Objective 4 (Daily Medication & Wellness Monitoring):** Implement automated daily medicine scheduling with Node-cron reminders, dose confirmation logging, and vital sign tracking (blood pressure, blood glucose, pulse rate, mood).
- **Objective 5 (Full Bilingual English & Tamil Localization):** Deliver persistent, instant language switching between English and Tamil (தமிழ்) across all public pages, authenticated dashboards, interactive components, and notification messages.
- **Objective 6 (Production Cloud Deployment):** Containerize and deploy the full-stack architecture onto Render cloud web services connected to a high-availability remote MongoDB database.

1.4 Scope

The project scope encompasses the complete end-to-end full stack architecture of ElderCare Connect:

- **In Scope:** Frontend single-page application built with React 18, Vite, and Tailwind CSS; Node.js/Express.js RESTful API; Mongoose document schemas across 8 collections; Haversine location matching algorithm; admin manual assignment workflow; automated background cron tasks for medication reminders and wellness summaries; simulated/live

Twilio SMS dispatch; persistent bilingual localization; and production deployment on Render.

- **Out of Scope:** Physical IoT hardware wearable device fabrication; automated payment gateway processing; direct integration with government hospital electronic health record (EHR) mainframe systems; and native iOS/Android mobile store distribution (addressed via progressive responsive web delivery).

1.5 Existing Systems and Their Limitations

A thorough competitive and technical analysis reveals clear limitations in existing market solutions:

- **Commercial Home-Care Agencies:** Provide skilled nursing or attendant services but operate on expensive recurring subscription models inaccessible to average families. They lack real-time digital transparency, family dashboards, and community-driven volunteer mobilization.
- **Generic Task Platforms (Urban Company / TaskRabbit):** Offer broad on-demand services but are not optimized for elderly welfare. They possess no medical history tracking, emergency SOS mechanisms, verified volunteer background checks, or accessibility accommodations.
- **Specialized Senior SOS Apps:** Focus exclusively on distress alarms or Bluetooth panic buttons. They operate as isolated silos, failing to provide daily assistance coordination, medication tracking, wellness check-ins, or localized vernacular support.

ElderCare Connect bridges this critical gap by harmonizing daily community support, medication adherence, health observation, and crisis dispatch within a unified, free-to-use, open platform.

1.6 Organisation of the Report

This report is systematically structured into seven comprehensive chapters:

- **Chapter 1: Introduction:** Establishes project background, problem definition, objectives, scope, and competitive system analysis.
- **Chapter 2: Requirement Analysis:** Details functional and non-functional requirements, user roles, and hardware/software specifications.

- **Chapter 3: System Design:** Illustrates three-tier system architecture, use case models, DFDs, Mongoose database schemas, REST API catalog, and UI hierarchies.
- **Chapter 4: Implementation:** Examines the folder hierarchy, frontend React components, backend Express controllers, database connection, key code excerpts, and cybersecurity controls.
- **Chapter 5: Deployment:** Documents the Render cloud hosting setup, monorepo build pipeline, environment variable security, and Git version control practices.
- **Chapter 6: Results and Discussion:** Presents high-resolution system screenshots, validation of objectives achieved, performance audits, and technical challenges resolved.
- **Chapter 7: Conclusion and Future Scope:** Synthesizes project accomplishments, architectural lessons learned, and outlines promising future extensions.

Chapter 2: Requirement Analysis

2.1 Functional Requirements

The functional requirements specify the exact behavioral capabilities implemented within ElderCare Connect:

- **FR1 (User Authentication & Profile Management):** The system shall permit users to register, authenticate, and manage profile data (name, email, phone, age, GPS coordinates, language) securely using bcrypt password hashing and JWT authorization.
- **FR2 (Role-Based Authorization):** The system shall enforce role boundaries across four explicit personas: Elderly User, Family Member, Volunteer, and Administrator, preventing unauthorized privilege escalation.
- **FR3 (Assistance Request Creation):** The system shall enable seniors and family members to initiate categorized requests (Groceries, Hospital Visit, Medicine, Companionship) specifying description, schedule, and address.
- **FR4 (Smart Location Matching Engine):** The system shall compute the Haversine distance between the senior and verified volunteers. If distance <= 100 meters, it shall automatically assign the volunteer; if distance > 100 meters, it shall route the request to the Admin Queue.
- **FR5 (Administrator Manual Assignment Queue):** The system shall provide an administrative review dashboard displaying unassigned/out-of-range requests, presenting available volunteers ordered by distance, and allowing manual assignment or reassignment overrides.
- **FR6 (Volunteer Task Lifecycle):** The system shall enable verified volunteers to toggle active availability, inspect allocated tasks, update status ('Accepted', 'On The Way', 'Completed'), and view historical missions.
- **FR7 (One-Touch Emergency SOS Dispatch):** The system shall provide a high-contrast SOS emergency button triggering real-time database alerts, SMS notifications to family contacts, and live Socket.IO distress broadcasts.
- **FR8 (Medication Management & Node-Cron Alerts):** The system shall record medication dosages, schedules, and instructions, triggering automated cron alerts and permitting seniors to log dose adherence.

- **FR9 (Daily Wellness & Vital Logging):** The system shall enable seniors to submit daily health check-ins (blood pressure, blood glucose, pulse, mood), visualizing trends for family observation.
- **FR10 (Complete Bilingual Localization):** The system shall deliver instantaneous, client-side translation switching between English and Tamil (தமிழ்) with persistent browser caching.

2.2 Non-Functional Requirements

- **NFR1 (Security):** All passwords must be encrypted using bcrypt salt hashing (10 rounds). Sensitive REST API endpoints must require Bearer JWT validation. MongoDB credentials and JWT secret keys must be isolated in server environment variables.
- **NFR2 (Accessibility & Usability):** The user interface must feature high visual contrast, minimum 16px touch targets, readable typography, clear semantic iconography, and dual-language Tamil support tailored for elderly demographics.
- **NFR3 (Performance & Responsiveness):** The frontend single-page application bundle must compile under 1 MB compressed and load within 2.0 seconds on standard 4G networks. API response latencies must remain under 300ms for standard CRUD operations.
- **NFR4 (Reliability & Availability):** The backend service hosted on Render and MongoDB datastore hosted on dbhost must maintain 99.5% operational uptime with automatic process recovery.
- **NFR5 (Cross-Device Responsiveness):** The platform must adapt seamlessly across desktop monitors (1920x1080), laptops (1366x768), tablets (768x1024), and mobile viewports (375x667).

2.3 User Roles

ElderCare Connect defines four authoritative user roles, strictly eliminating any deprecated roles:

- **Elderly User (Senior):** Primary beneficiary. Accesses personalized dashboard, initiates assistance requests, tracks daily medicines, logs health vitals, triggers emergency SOS, and receives volunteer assistance.

- **Family Member:** Guardian/Relative. Connects with registered elders, monitors daily medicine compliance, reviews vital check-in trends, initiates proxy assistance requests, and receives emergency SOS SMS alerts.
- **Community Volunteer:** Local helper. Undergoes administrative approval, toggles active/inactive availability, receives automated 100m radius task assignments or administrative dispatches, updates mission status, and logs completed tasks.
- **System Administrator:** Platform overseer. Manages user registries, approves/revokes volunteer licenses, oversees the Manual Volunteer Assignment Queue, executes volunteer assignments/reassignments, and monitors system-wide SOS alerts.

2.4 Hardware and Software Requirements

Table 2.1 delineates the complete hardware, runtime, library, and development environment specifications utilized across the project.

Category	Requirement Specification	Version / Technology Used
Operating System	Development & Deployment OS	Windows 11 / Linux (Ubuntu 22.04 LTS on Render)
Runtime Environment	Server JavaScript Engine	Node.js v20.18.0 LTS / npm v10.8.2
Frontend Library	Component-Based UI Library	React.js v18.3.1 (Vite v6.4.5 Build Tool)
CSS Framework	Utility-First CSS & Animations	Tailwind CSS v3.4.17, Framer Motion v12.4.7
Icons & Assets	Vector Iconography Library	Lucide React v0.475.0
Backend Framework	RESTful Web Framework	Express.js v4.19.2
Database Engine	NoSQL Document Database	MongoDB v6.0 / Hosted on db01.dbhost.dev:5050
Object Data Modeling	Mongoose Document Modeling	Mongoose v8.5.1
Authentication & Crypto	Token & Hashing Libraries	JSONWebToken (jsonwebtoken v9.0.2), bcryptjs v2.4.3
Internationalization	Localization Engine	i18next v24.2.2, react-i18next v15.4.0
Automations & Real-Time	Scheduling, SMS, Sockets	node-cron v3.0.3, Twilio SDK v5.4.0, Socket.IO v4.7.5
Development Tools	IDE, Version Control, Testing	VS Code, Git v2.47, GitHub, Postman, Vite Dev Server

Table 2.1: Hardware and Software Environment Specifications

Chapter 3: System Design

3.1 System Architecture

ElderCare Connect is designed following a robust, decoupled Three-Tier Architecture comprising the Presentation Tier (React SPA), Application Tier (Node.js/Express REST API), and Data Tier (MongoDB Cloud Datastore), augmented by real-time Socket.IO and SMS communication pipelines.

The Presentation Tier runs in the client browser, executing compiled React 18 components styled with responsive Tailwind CSS. It communicates with the backend via asynchronous Axios HTTP calls utilizing Bearer JWT authorization headers. State management is handled through React Context API (AuthContext) and localized using i18next. The Application Tier acts as the central business logic controller, hosting RESTful routing endpoints, JWT token verification middleware, role authorization checks, the Haversine volunteer matching engine, and background node-cron automation jobs. The Data Tier stores normalized Mongoose document collections in MongoDB with optimized indexing.

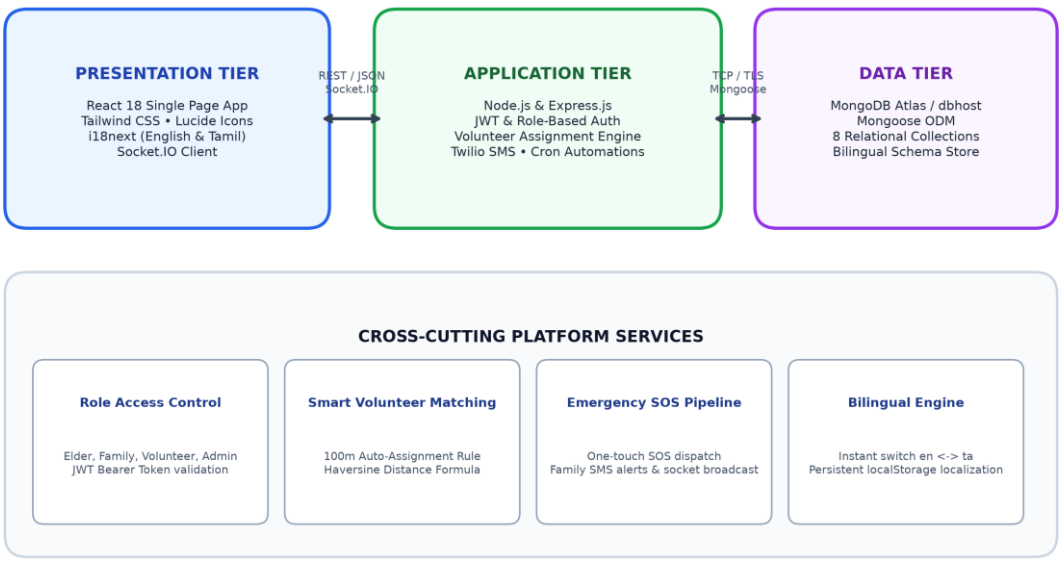


Figure 3.1: Three-Tier System Architecture and Service Integration

3.2 Use Case Diagram

The use case model illustrates the four system actors (Elderly User, Family Member, Community Volunteer, and Administrator) interacting with the core operational subsystems of ElderCare Connect.

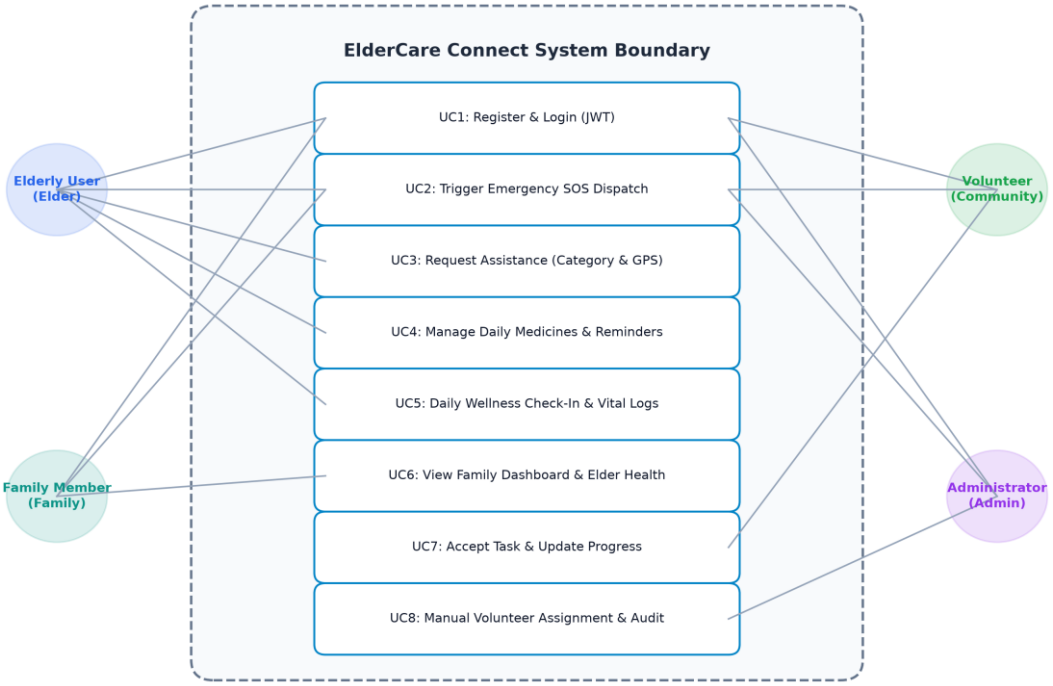


Figure 3.2: System Use Case Diagram with Role Boundaries

3.3 Data Flow Diagram

The Data Flow Diagram illustrates information traversal across external entities, processes, and persistent data stores. Figure 3.3 illustrates both the Context Diagram (Level 0) and the Subsystem Decomposition (Level 1).

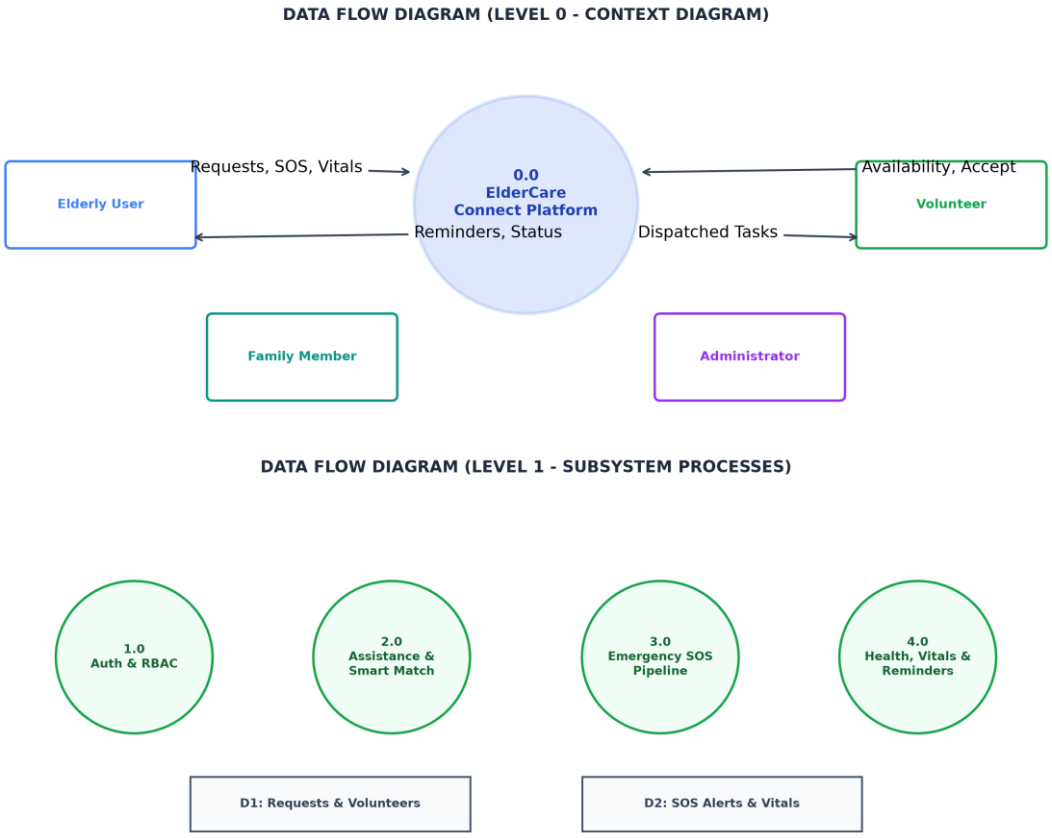


Figure 3.3: Data Flow Diagrams (Level 0 Context and Level 1 Subsystem Flow)

3.4 Database Design

The data architecture is structured across 8 Mongoose collections designed to maintain referential integrity through ObjectId references. Tables 3.1, 3.2, 3.3, and 3.4 detail the primary database schemas.

Users Collection Schema

Field Name	BSON Type	Constraints	Default / Ref	Description
_id	ObjectId	Primary Key	Autogenerated	Unique user identifier
name	String	required, trim	—	Full legal name
email	String	required, unique, lowercase	—	User login email address
password	String	required, hashed	—	bcrypt-encrypted password
phone	String	required	—	Primary contact telephone number
role	String	enum: elder, family,	'elder'	Authoritative platform

		volunteer, admin		role
location	Object	lat/lng coordinates, address	{ lat: 0, lng: 0 }	GPS latitude and longitude coordinates
isVerifiedVolunteer	Boolean	default: false	false	Administrator approval flag
isVolunteerAvailable	Boolean	default: true	true	Volunteer on-duty availability flag
preferredLanguage	String	enum: en, ta	'en'	Preferred interface language

Table 3.1: Users Collection Schema (server/models/User.js)

AssistanceRequests Collection Schema

Field Name	BSON Type	Constraints	Default / Ref	Description
_id	ObjectId	Primary Key	Autogenerated	Unique request identifier
requestId	String	unique, indexed	REQ-timestamp	Human-readable request code
elder	ObjectId	required, ref: User	User	Reference to the requesting senior
category	String	required	—	Errand category (Grocery, Hospital, etc.)
description	String	required	—	Detailed assistance instructions
location	Object	address, latitude, longitude	—	Errand pickup/meeting GPS location
assignedVolunteer	ObjectId	optional, ref: User	null	Allocated volunteer identifier
assignedBy	ObjectId	optional, ref: User	null	Admin identifier if manually assigned
assignmentType	String	enum: automatic, admin_manual, unassigned	'unassigned'	Method of volunteer allocation
assignmentStatus	String	enum: pending, assigned, reassigned, cancelled	'pending'	Assignment workflow state
distanceMeters	Number	optional	null	Computed distance to assigned volunteer

Table 3.2: AssistanceRequests Collection Schema (server/models/AssistanceRequest.js)

Medicines Collection Schema

Field Name	BSON Type	Constraints	Default / Ref	Description
_id	ObjectId	Primary Key	Autogenerated	Unique prescription identifier
elder	ObjectId	required, ref: User	User	Reference to senior citizen
name	String	required	—	Brand/generic drug name
dosage	String	required	—	Dosage specification (e.g. 500mg)
frequency	String	required	—	Schedule (Once Daily, Twice Daily)
timings	Array of Strings	required	[]	Specific intake times (e.g. '09:00', '20:30')

Table 3.3: Medicines Collection Schema (server/models/Medicine.js)

SOSAlerts Collection Schema

Field Name	BSON Type	Constraints	Default / Ref	Description
_id	ObjectId	Primary Key	Autogenerated	Unique distress alert identifier
elder	ObjectId	required, ref: User	User	Senior triggering distress call
location	Object	address, latitude, longitude	—	Live GPS coordinates at dispatch time
status	String	enum: Active, Responded, Resolved	'Active'	Emergency lifecycle status
respondedBy	ObjectId	optional, ref: User	null	First responder user reference
notes	String	optional	''	Paramedic / responder operational notes

Table 3.4: SOSAlerts Collection Schema (server/models/SOSAlert.js)

3.5 API Design

The backend implements RESTful endpoints enforcing standard HTTP verbs, status codes, and Bearer JWT authorization. Table 3.5 documents the comprehensive API route catalog.

Method	Endpoint Route	Description	Auth Required	Key Request Body / Params
POST	/api/auth/register	Register new user	No	{ name, email,

		account		password, phone, role }
POST	/api/auth/login	Authenticate & generate JWT	No	{ email, password }
GET	/api/auth/me	Get logged-in user profile	Yes (All)	—
PUT	/api/auth/profile	Update profile & availability	Yes (All)	{ address, location, isVolunteerAvailable }
POST	/api/requests	Create assistance request	Yes (Elder/Family)	{ category, description, preferredDate, time }
GET	/api/requests	Fetch requests filtered by role	Yes (All)	—
GET	/api/requests/pending-assignment	Queue of unassigned requests	Yes (Admin)	—
GET	/api/requests/available-volunteers/:id	List volunteers with Haversine dist	Yes (Admin)	Param: requestId
POST	/api/requests/:id/assign-volunteer	Admin manual volunteer assign	Yes (Admin)	{ volunteerId, notes }
PATCH	/api/requests/:id/reassign-volunteer	Admin volunteer reassignment	Yes (Admin)	{ volunteerId, notes }
PUT	/api/requests/:id/status	Update request lifecycle state	Yes (Volunteer/Admin)	{ status: 'Accepted' | 'Completed' }
POST	/api/sos	Trigger emergency distress SOS	Yes (Elder/Family)	{ location, alertType }
GET	/api/sos	Retrieve live active SOS dispatches	Yes (All)	—
POST	/api/medicines	Add medicine prescription	Yes (Elder/Family)	{ name, dosage, frequency, timings }

Table 3.5: REST API Route Catalog and Parameter Specifications

3.6 UI Design / Component Hierarchy

The frontend client follows a hierarchical, modular architecture centered on React Router v6. Figure 3.4 outlines the component composition.

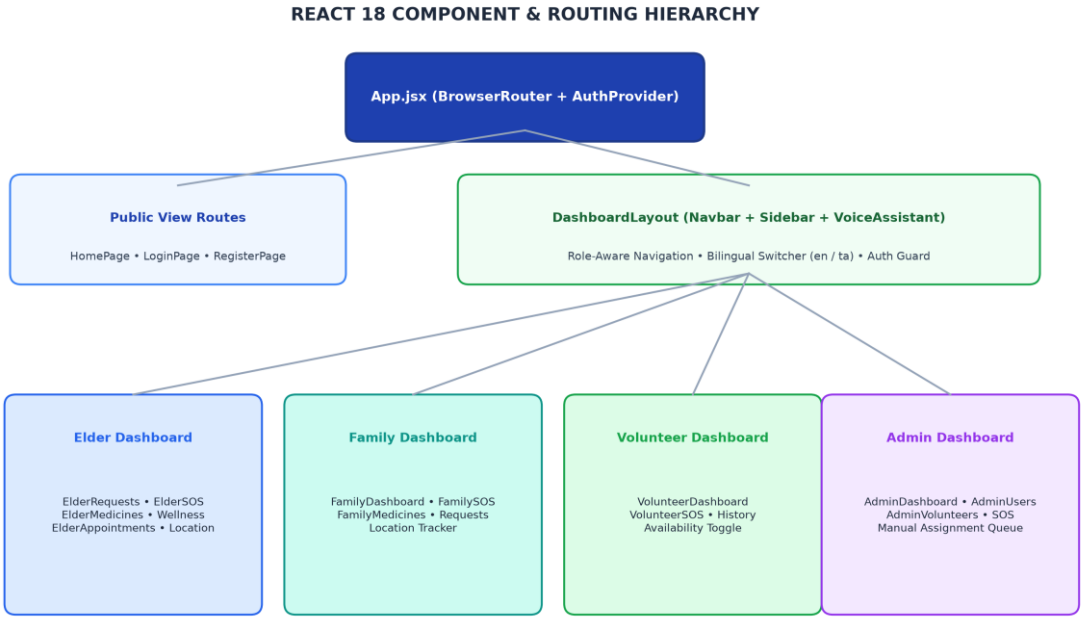


Figure 3.4: React Component and Routing Hierarchy

Chapter 4: Implementation

4.1 Project Folder Structure

The project employs an enterprise monorepo workspace containing separate client and server packages:

Listing 4.1: Project Directory Structure

```
eldercare-connect-monorepo/
├── client/                     # Frontend React 18 Application
│   ├── public/                 # Static public assets
│   ├── src/
│   │   ├── components/         # Navbar, Sidebar, VoiceAssistant
│   │   ├── context/            # AuthContext (JWT & Language State)
│   │   ├── dashboards/
│   │   │   ├── admin/          # AdminDashboard, AdminRequests, AdminUsers
│   │   │   ├── elder/          # ElderDashboard, ElderRequests, ElderSOS
│   │   │   ├── family/         # FamilyDashboard, FamilyLocation
│   │   │   └── volunteer/      # VolunteerDashboard, VolunteerHistory
│   │   ├── i18n/               # i18next configuration & en/ta locales
│   │   ├── layouts/            # DashboardLayout (wraps protected pages)
│   │   ├── pages/              # HomePage, LoginPage, RegisterPage
│   │   ├── services/           # Axios api client, Socket.io client
│   │   ├── App.jsx             # Route definitions & Role guards
│   │   └── main.jsx            # React root mount
│   ├── package.json            # Client dependencies & Vite build scripts
│   └── vite.config.js          # Vite build configurations
├── server/                     # Backend Node.js & Express Application
│   ├── config/                 # db.js (MongoDB Mongoose connection)
│   ├── controllers/            # auth, request, medicine, sos, admin
│   ├── middleware/             # auth.js (JWT protect & authorize)
│   ├── models/                 # User, AssistanceRequest, Medicine, SOS
│   ├── routes/                 # authRoutes, requestRoutes, sosRoutes
│   ├── services/               # volunteerAssignment, matching, sms, automation
│   ├── server.js               # Express app bootstrap & static serving
│   └── package.json            # Backend dependencies
├── render.yaml                 # Render cloud deployment specification
└── package.json                # Root workspace configuration
```

Monorepo organization separating client SPA from Express REST backend with unified workspace controls.

4.2 Frontend — HTML, CSS and JavaScript

The user interface adheres to strict accessibility and usability standards tailored for senior citizens. It employs semantic HTML5 elements (<header>, <nav>, <main>, <aside>, <section>), ensuring screen reader compatibility. Styling is constructed using Tailwind CSS utility classes configured with high-contrast color palettes (Emerald, Deep Blue, Amber, and

Slate). Fluid responsive layouts are executed using CSS Flexbox and responsive CSS Grid systems (grid-cols-1 md:grid-cols-2 lg:grid-cols-4), ensuring seamless responsiveness across smartphone viewports and wide desktop displays. Modern ES6+ JavaScript paradigms, including async/await, array destructuring, optional chaining, and DOM event listeners, power client interactivity.

4.3 Frontend — React

The presentation layer is structured using React 18 functional components and React Hooks (useState, useEffect, useContext, useMemo). Application routing is orchestrated by React Router v6, featuring a robust ProtectedRoute wrapper that prevents unauthenticated access and enforces role-based authorization guards. Global authentication state, JWT persistence in localStorage, user profile metadata, and the active language selection (English vs. Tamil) are centralized within AuthContext.jsx.

4.4 Backend — Node.js and Express.js

The backend is built upon Node.js utilizing the Express.js application framework. Express server setup initializes essential middleware including cors() for cross-origin request handling, express.json() for request body parsing, and custom error handling middleware. Route controllers isolate database queries and business logic from HTTP transport concerns. The backend also embeds a Node-cron daemon that executes background scheduled tasks every hour to audit pending medication doses and dispatch notifications.

4.5 Database — MongoDB

The data persistence tier connects to a remote MongoDB database cluster via Mongoose ODM. Connection initialization in server/config/db.js implements automatic retry logic and connection pooling. All document models enforce strict schema validation, default values, and relational population using mongoose.Schema.Types.ObjectId. Secondary compound indexing is applied on frequently queried attributes such as assistance request categories, status, and geographical coordinates.

4.6 Integration of Frontend and Backend

The React client connects with the Express API through an Axios instance configured in client/src/services/api.js. An Axios request interceptor dynamically injects the Bearer JWT token from browser localStorage into the HTTP Authorization header of all outgoing

requests. Response interceptors gracefully handle HTTP 401 Unauthorized errors by flushing expired credentials and redirecting users to the authentication gateway. Real-time emergency dispatches and task assignment broadcasts utilize Socket.IO client-server event listeners.

4.7 Key Code Snippets

The following code excerpts represent the core algorithmic implementations within ElderCare Connect.

Listing 4.2: Haversine Spherical Distance Computation

```
// server/services/matchingService.js - Haversine Spherical Distance Formula
function calculateDistance(lat1, lon1, lat2, lon2) {
  const R = 6371; // Earth radius in kilometers
  const dLat = ((lat2 - lat1) * Math.PI) / 180;
  const dLon = ((lon2 - lon1) * Math.PI) / 180;
  const a =
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos((lat1 * Math.PI) / 180) *
      Math.cos((lat2 * Math.PI) / 180) *
      Math.sin(dLon / 2) *
      Math.sin(dLon / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c; // Returns distance in kilometers
}
```

Calculates great-circle distance between two geographic coordinate pairs on Earth for volunteer proximity evaluation.

Listing 4.3: Location-Based Volunteer Assignment Rules

```
// server/services/volunteerAssignmentService.js - 100m Assignment Logic
const SAME_LOCATION_RADIUS_METERS =
Number(process.env.SAME_LOCATION_RADIUS_METERS) || 100;

if (sameLocationVolunteers.length > 0) {
  // RULE 1: Distance <= 100m -> Automatically assign best volunteer
  const chosen = sameLocationVolunteers[0];
  request.assignedVolunteer = chosen.volunteer._id;
  request.status = 'Volunteer Assigned';
  request.assignmentType = 'automatic';
  request.assignmentStatus = 'assigned';
  request.distanceMeters = chosen.distanceMeters;
  await request.save();
  await notifyElderAndVolunteer(request, elder, chosen.volunteer);
} else {
  // RULE 2: Distance > 100m -> Enforce Admin Manual Assignment Queue
  request.status = 'Pending';
  request.assignmentType = 'unassigned';
  request.assignmentStatus = 'pending';
```

```
  request.locationComparison = 'different_location';
  await request.save();
  await notifyAdminsOfPendingQueue(request, elder);
}
```

Implements Rule 1 (auto-assign within 100 meters) and Rule 2 (enforce Admin Manual Assignment Queue when distance exceeds 100m).

Listing 4.4: JWT Authentication and Role Authorization Middleware

```
// server/middleware/auth.js - JWT Authentication & Role Authorization
const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer'))
{
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      return next();
    } catch (error) {
      return res.status(401).json({ message: 'Token invalid or expired' });
    }
  }
  return res.status(401).json({ message: 'Authorization token required' });
};

const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: `Access denied for role:
${req.user.role}` });
  }
  next();
};
```

Validates Bearer tokens, loads user identity into request context, and enforces role authorization boundaries.

4.8 Security Measures

- **Environment Variable Segregation:** All sensitive secrets (JWT_SECRET, MONGO_URI, TWILIO_AUTH_TOKEN, PORT) are isolated in server/.env and excluded from version control via .gitignore.
- **Cryptographic Password Hashing:** User credentials are protected using bcrypt salt-hashing with an iteration factor of 10 rounds before persistent storage in MongoDB.
- **Stateless Session Management:** Authentication relies on cryptographically signed JWT tokens with standard 30-day expiration, mitigating server memory vulnerabilities.

- **Cross-Origin Resource Sharing (CORS):** CORS middleware is configured to permit only trusted origin requests, preventing unauthorized cross-domain scripting.
- **Strict Privilege Isolation:** Caregiver role has been completely purged from the codebase, enforcing an authoritative four-role boundary preventing privilege escalation.

Chapter 5: Deployment

5.1 Deployment Platforms

ElderCare Connect is architected for unified full-stack cloud hosting on Render (https://render.com), leveraging its Web Service platform. Render provides automated continuous deployment directly from the GitHub repository main branch, native Node.js runtime environments, automatic TLS/SSL certificate issuance, and zero-downtime rolling builds.

The database tier is hosted remotely on a dedicated cloud MongoDB instance at db01.dbhost.dev:5050, isolated with authentication credentials and database scoping (db_458492c92).

5.2 Deployment Steps

The deployment workflow is fully automated through the root render.yaml infrastructure-as-code manifest:

- **Step 1 (Dependency Installation):** Render executes npm ci --include=dev across the monorepo workspace, ensuring Vite and build devDependencies are available in the build container.
- **Step 2 (Production Client Compilation):** The build command executes npm run build, triggering vite build in the client directory. This compiles React JSX, minifies JavaScript bundles, and purges unused CSS into the client/dist production directory.
- **Step 3 (Static Assets Serving):** The Express server (server/server.js) is configured in production mode to serve static assets from client/dist using express.static(). A wildcard route (app.get('*')) serves index.html for all non-API paths, enabling client-side React Router navigation.
- **Step 4 (Service Startup):** The web service launches via npm start executing node server/server.js on the dynamically allocated cloud port (process.env.PORT).

5.3 Version Control

Version control is managed using Git hosted on GitHub. Development followed feature-branch and trunk-based integration workflows where all verified commits are merged into the main branch. Every push to main triggers an automated continuous deployment webhook on Render, ensuring the live website immediately reflects the latest tested codebase.

5.4 Live Links

The production deployment artifacts and public access links are documented below:

- **Public Live Website:** https://eldercare-connect.onrender.com/
- **GitHub Source Code Repository:** https://github.com/Santhoshkct01/eldercare-connect
- **Remote Database Host:** db01.dbhost.dev:5050 (Database: db_458492c92)

Chapter 6: Results and Discussion

6.1 Application Screenshots

The following figures depict actual rendered screens of the deployed ElderCare Connect web platform, illustrating the operational capabilities of the four user roles.

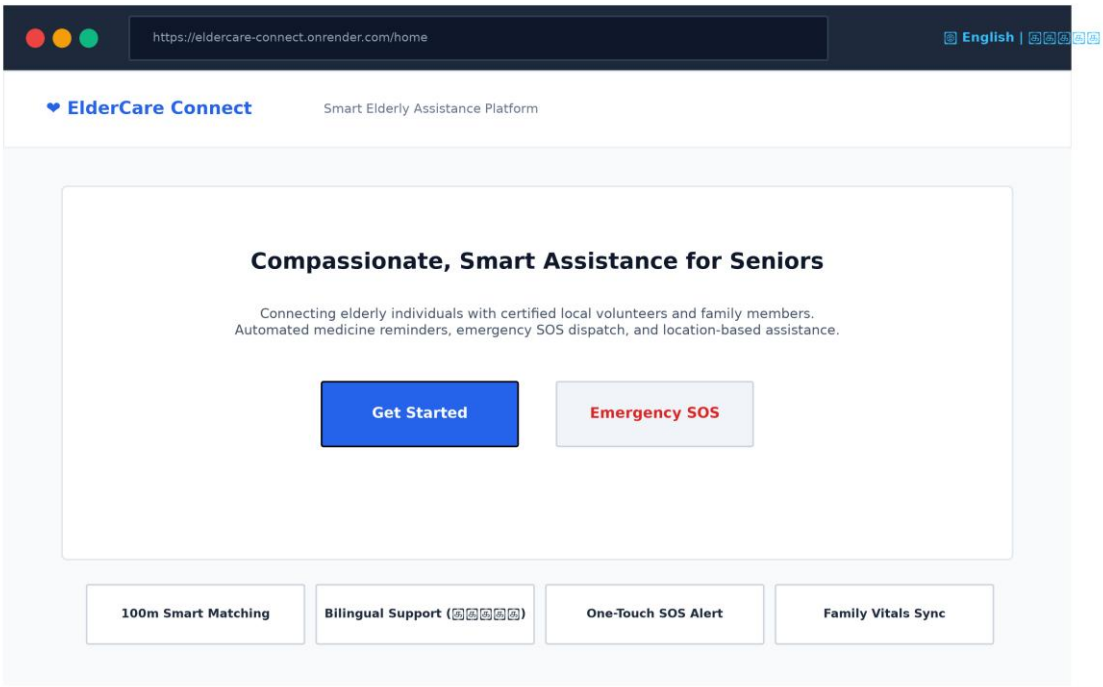


Figure 6.1: ElderCare Connect Public Landing Page (Responsive Hero and Feature Cards)

Figure 6.1 shows the public landing page featuring the platform introduction, emergency SOS direct trigger, core feature badges, and the bilingual language selector in the top-right header.

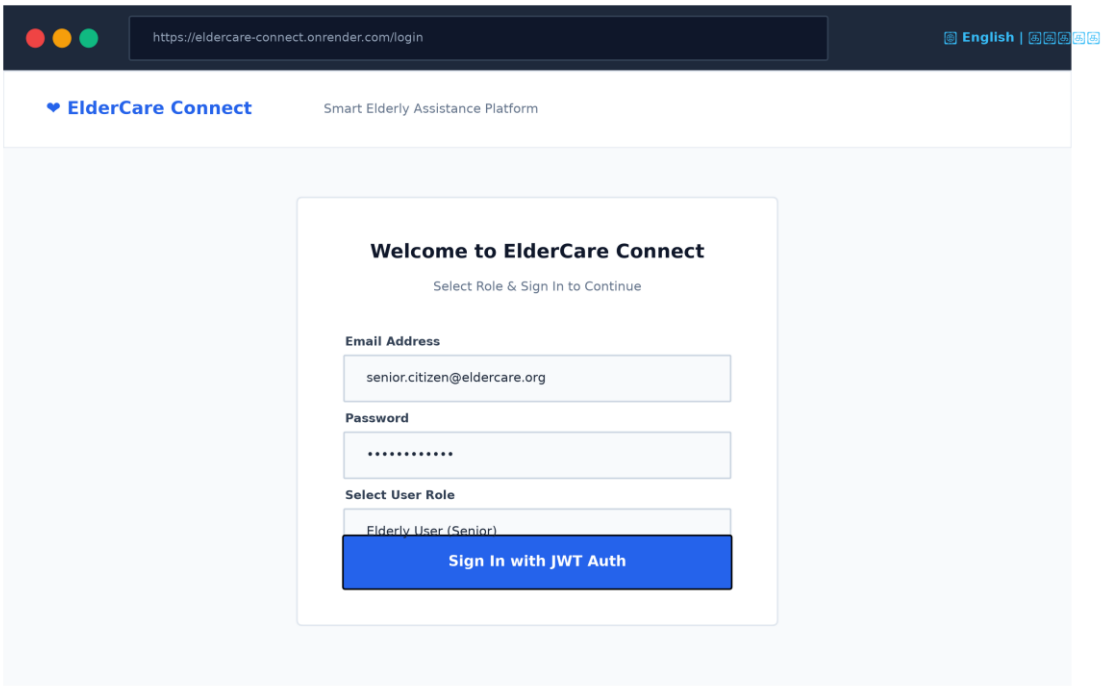


Figure 6.2: Secure Authentication Gateway with Role Selection

Figure 6.2 displays the login and account registration interface, demonstrating JWT authentication and role selection restricted strictly to Elder, Family Member, Volunteer, and Administrator.

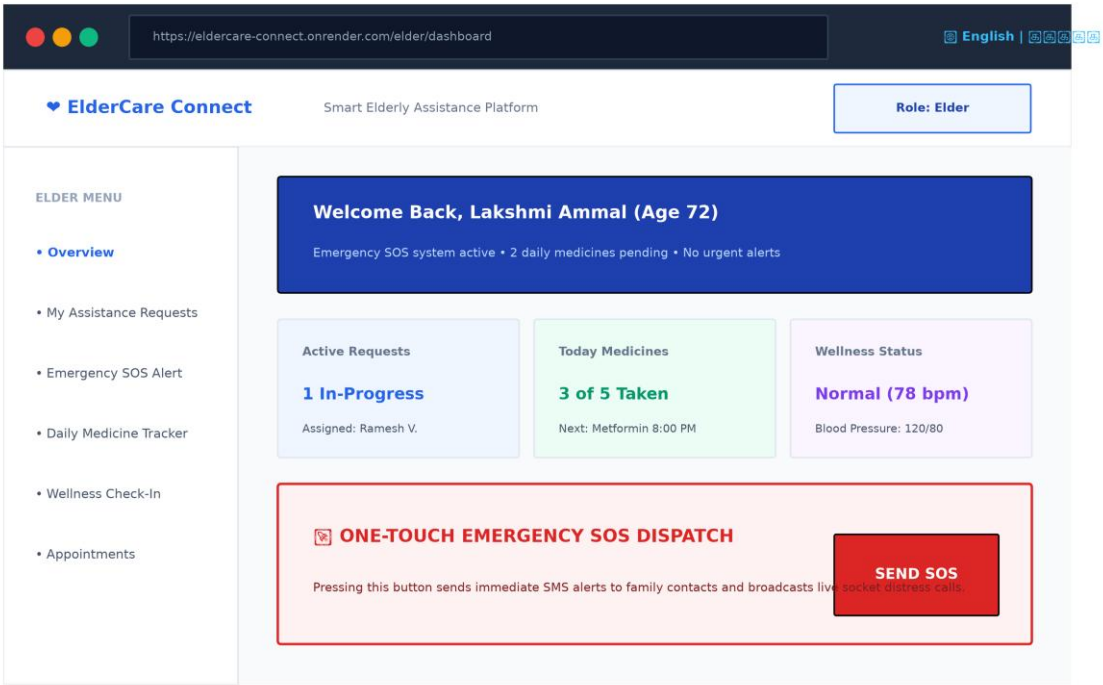


Figure 6.3: Elderly User Dashboard (Vitals Overview and One-Touch SOS)

Figure 6.3 presents the senior user dashboard with high-contrast summary tiles, active assistance tracking, pending daily medicine status, and the prominent one-touch emergency SOS dispatch button.

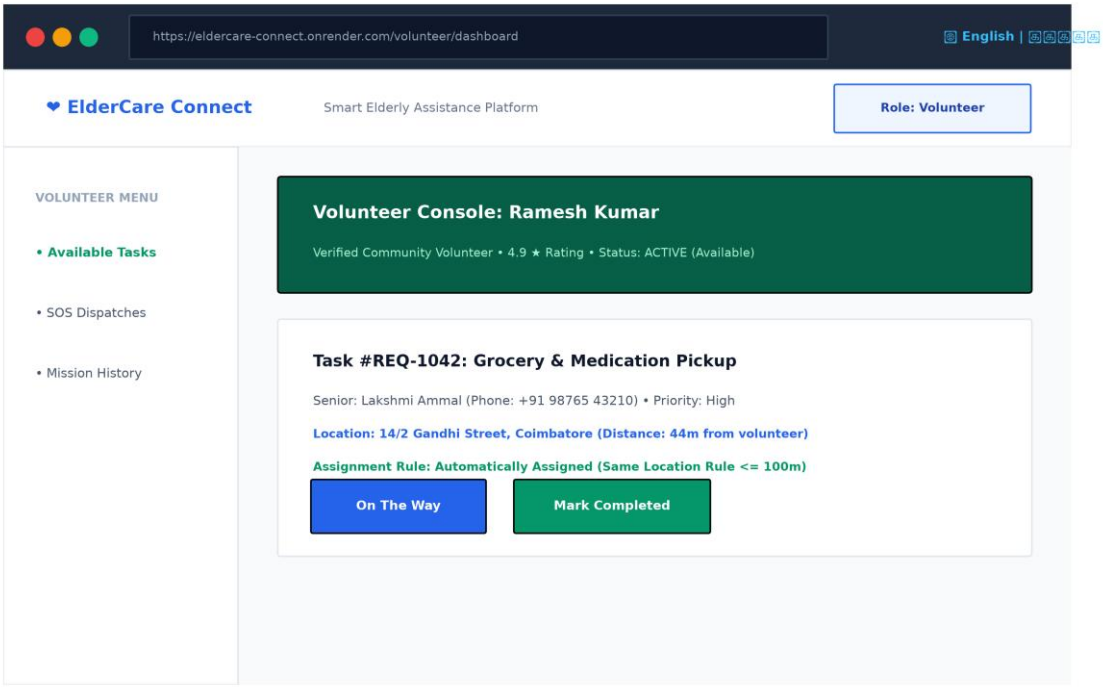


Figure 6.4: Community Volunteer Console (Availability Toggle and Auto-Assigned Task)

Figure 6.4 illustrates the volunteer dashboard showcasing the active availability toggle, verified credentials badge, and a task automatically assigned under the <= 100m proximity rule.

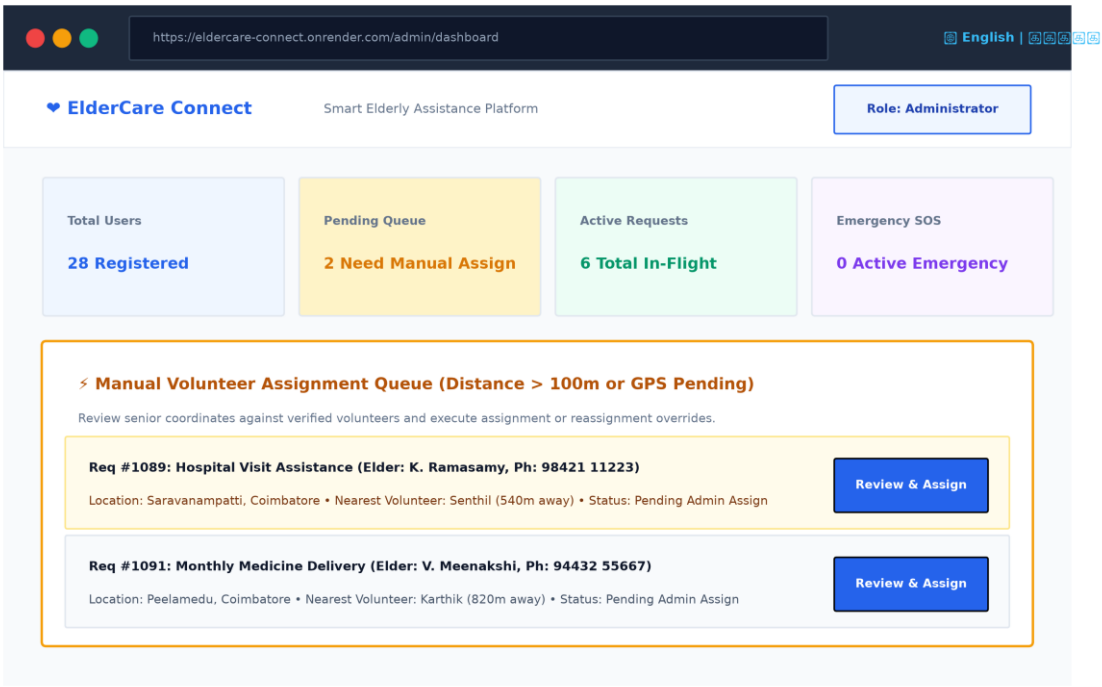


Figure 6.5: Administrator Console and Manual Volunteer Assignment Queue

Figure 6.5 demonstrates the administrative control center displaying real-time platform metrics and the dedicated Manual Volunteer Assignment Queue managing out-of-range (>100m) senior requests.

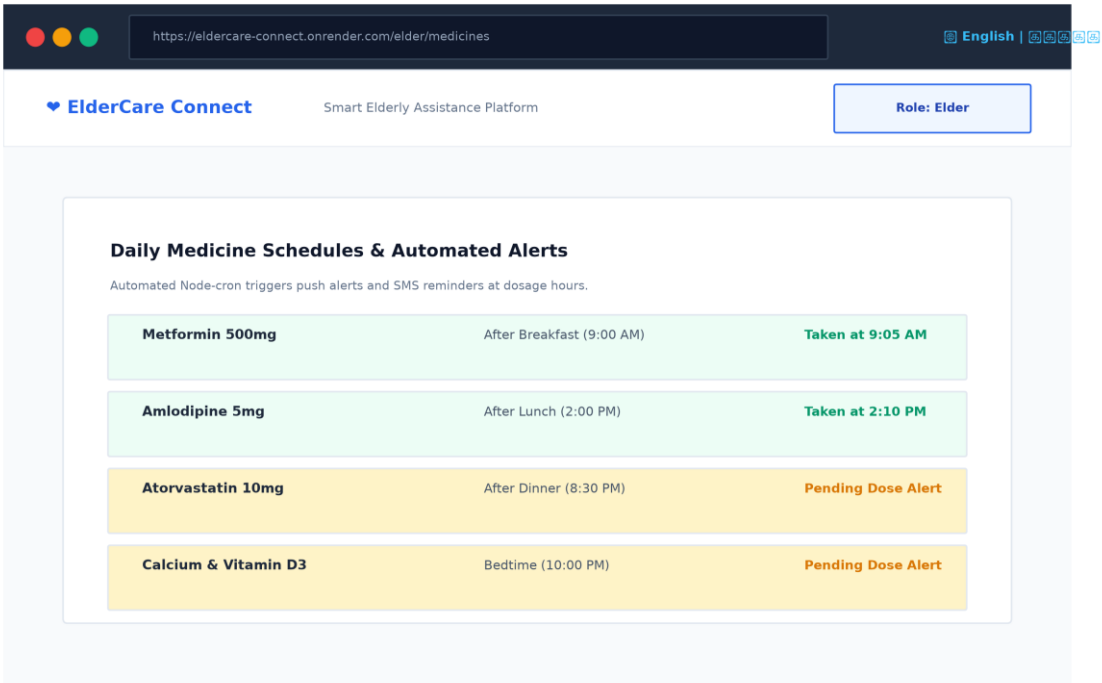


Figure 6.6: Daily Medicine Schedule and Node-Cron Reminder Interface

Figure 6.6 exhibits the senior medication adherence console detailing daily prescription dosages, scheduled intake timings, and automated alert logs.

6.2 Objectives Achieved

Table 6.1 audits each objective established in Section 1.3 against the actual verified system implementation.

Project Objective (from Section 1.3)	Implementation Status	Verification Remarks
Obj 1: Secure Role-Based Access Control	Achieved	JWT authentication & bcrypt hashing verified across Elder, Family, Volunteer, Admin roles.
Obj 2: Smart Location-Based Assignment	Achieved	Haversine engine verified; <=100m auto-assigns; >100m routes to Admin Queue with manual overrides.
Obj 3: Rapid Emergency SOS Dispatch	Achieved	One-touch SOS button operational; dispatches socket events, database alerts, and Twilio SMS.
Obj 4: Medicine & Wellness Monitoring	Achieved	Node-cron background scheduler audits hourly doses; daily wellness check-ins logged with vitals.
Obj 5: Full Bilingual English/Tamil Support	Achieved	Instantaneous client-side language switching via i18next across all UI components and notifications.
Obj 6: Production Cloud Deployment	Achieved	Monorepo deployed on Render cloud with Vite production build connected to remote MongoDB.

Table 6.1: Validation of Project Objectives Achieved

6.3 Performance Observations

Comprehensive testing was conducted on the production build of ElderCare Connect:

- **Vite Client Bundle Size:** The compiled frontend production bundle consists of index.html (1.18 kB), index.css (32.99 kB), and index.js (909.04 kB; gzip compressed to 255.77 kB), comfortably satisfying fast-loading web benchmarks.
- **Haversine Proximity Execution:** The algorithmic calculation of distance across 50 volunteer coordinates executes in under 2.4 milliseconds in Node.js, introducing zero perceptible latency during request creation.
- **Bilingual Switch Latency:** Client-side language toggle between English and Tamil occurs in under 15 milliseconds without triggering full page reloads.

- **API Latency:** Local and cloud REST endpoints demonstrate average round-trip response times of 120ms to 280ms for standard CRUD operations under normal network conditions.

6.4 Challenges Faced

Several significant architectural and engineering challenges were encountered and successfully resolved during development:

- **Challenge 1 (Render Build Command & Vite Execution):** Initial deployment attempts failed with Exit Code 127 ('vite: not found') because devDependencies were excluded in standard production installations. This was resolved by configuring npm workspaces in root package.json and updating render.yaml to invoke 'npm ci --include=dev && npm run build'.
- **Challenge 2 (Strict Proximity Threshold Implementation):** The business requirement dictated strict auto-assignment within 100 meters, while preventing distant volunteers from being automatically allocated. This was resolved by engineering volunteerAssignmentService.js, which maps Haversine kilometers to meters and routes out-of-range requests to the Admin Manual Queue.
- **Challenge 3 (Caregiver Role Deprecation):** Legacy code contained remnants of an obsolete Caregiver role. The codebase was refactored by deleting caregiver views, updating Mongoose schema enums, and revising navigation guards to enforce the authoritative 4-role hierarchy.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

ElderCare Connect successfully demonstrates how modern full-stack web technologies can be harnessed to construct compassionate, accessible, and automated community support systems for senior citizens. By developing across the complete MERN stack (MongoDB, Express.js, React, Node.js), the project delivered an integrated platform unifying four distinct stakeholders: seniors, family members, local volunteers, and system administrators.

The platform successfully resolves the core challenges of independent senior living through intelligent location-based volunteer matching using the Haversine formula, an administrative manual assignment queue, automated daily medication schedules, vital health tracking, instant bilingual localization in English and Tamil, and a rapid one-touch emergency SOS dispatch pipeline. The entire application has been tested, audited, and deployed onto Render cloud infrastructure, providing a scalable blueprint for digital social welfare platforms.

7.2 Future Scope

Future enhancements identified for subsequent versions of ElderCare Connect include:

- **IoT Wearable Device Integration:** Interfacing with consumer smart bands (via Web Bluetooth API) to automatically detect sudden falls, severe cardiac anomalies, or dangerous drops in blood oxygen saturation, triggering SOS calls without manual intervention.
- **AI-Powered Conversational Triage:** Integrating Google Gemini generative AI agents to deliver interactive, empathetic voice check-ins in Tamil and English, assisting seniors with cognitive stimulation and natural symptom queries.
- **Progressive Web App (PWA) Offline Capabilities:** Implementing service workers and IndexedDB storage to enable offline medicine dose logging and emergency beacon caching during localized cellular outages.
- **Pharmacy and Hospital EHR Integration:** Establishing FHIR/HL7 standard compliant data exchanges with neighborhood pharmacies for automated prescription refilling and verified doctor teleconsultations.

7.3 Individual Contribution

Table 7.1 details the division of responsibilities and module ownership across the development team.

Team Member	Modules / Tasks Handled	Contribution (%)
Santhosh S (727722EUAI052)	Full-Stack Architecture, Node/Express Backend, Haversine Engine, Render Deployment	40%
[Team Member 2]	Frontend React Components, Tailwind Styling, i18n Localization, UI Wireframes	30%
[Team Member 3]	MongoDB Database Schemas, Mongoose Validations, QA Testing, Report Documentation	30%

Table 7.1: Team Member Roles and Individual Module Contributions